System Design: Monolithic Architecture

Easy lecture notes | Simple English | Java Spring Boot examples

What this lecture covers: architecture, monolithic applications, their advantages and disadvantages, and the difference between a website and a web application.

1. What is software architecture?

Architecture is the internal plan of an application. Before building software, we decide which parts it needs, how those parts communicate, and where the data is stored.

Example: Before building an online shopping app, we plan Login, Products, Cart, Orders and Payments.

2. What is monolithic architecture?

A monolithic application keeps its main features or modules together in one application that is usually built and deployed as a single unit.

Simple example: A Spring Boot project contains Login, Products, Orders and Payments in the same deployable application.

ONE SPRING BOOT APPLICATION
Login | Products | Orders | Payments
Controllers -> Services -> Repositories
Database (connected to the application)

Important clarification: A monolith does not require the frontend, backend and database to run in the same process or on the same server. The main idea is that application functionality is tightly packaged and deployed together. The lecture uses the simplified "all together" explanation.

3. Common parts of a web application

Frontend: What users see (HTML, CSS, Angular).
Backend: Business logic (Java, Spring Boot).
Data storage: Stores information (MySQL, PostgreSQL, Oracle).

4. Advantages of a monolith

Easy to start: One project is often simpler for a small team.
Simple local testing: Modules can be tested together.
Fewer network calls: Internal module calls usually do not need remote service requests.
Straightforward initial deployment: Often one main application to deploy.
Centralized security setup: Security can be configured in one application, though it is not automatically more secure.

5. Disadvantages of a monolith

Redeployment: A change in one module usually requires rebuilding/redeploying the application.
Scaling: Scaling only one busy module is harder; often the whole app is replicated.
Large codebase: As features grow, development and maintenance can become difficult.

Technology flexibility: Using different languages for separate modules is less straightforward.
Failure impact: A serious issue in one part may affect the whole application.

6. Simple Java Spring Boot example

Imagine one project with these packages:

```
com.shop
  login/
  product/
  order/
  payment/
  ShopApplication.java
```

All modules run inside the same Spring Boot application. You may deploy one JAR file such as shop.jar.

7. Monolith vs microservices (extra context)

Monolith	Microservices
One main deployable application	Multiple independently deployed services
Simpler to begin with	More operational complexity
Often scale whole application	Can scale services separately
Internal method calls common	Network/API calls between services common

8. Website vs web application

The lecture describes a website as mainly "read only" and a web application as interactive. This is a beginner-friendly shortcut, not a strict technical rule. Modern websites can be interactive too. A web application usually emphasizes user actions, workflows and changing data (for example, posting content or placing an order).

9. Interview questions: quick answers

Q1. What is monolithic architecture?
A software architecture where major features are built and deployed together as one application.

Q2. Give one advantage.
It is usually easier to build and deploy initially.

Q3. Give one disadvantage.
Even a small change may require redeploying the entire application.

Q4. Is a monolith always bad?
No. It can be a good choice for smaller products, simple deployments and teams that want lower operational complexity.

Q5. Can a monolith use a separate database?
Yes. The database can run on a separate server.

10. One-minute revision

Architecture = Internal application design.

Monolith = Many features in one deployable application.

Good for = Simple starting point and fewer moving parts.

Challenges = Whole-app redeployment, scaling and growing complexity.

Source note: Based on the Hindi transcript you supplied (about 9 minutes). Several phrases were unclear or incorrectly auto-transcribed. The notes preserve the clear lecture themes, with technical clarifications and a small microservices comparison marked as extra context.